

```

from pathlib import Path
import json
import pandas as pd
import re

data_root = Path("./data/2024/03")

json_files = list(data_root.rglob("*.json"))
print(f"Fichiers trouvés dans tous les dossiers : {len(json_files)}")

def extract_all_data_merged(files):
    all_calls = []
    for f_path in files:
        try:
            with open(f_path, 'r', encoding='utf-8') as f:
                data = json.load(f)

                segments = data.get('results', {}).get('audio_segments',
[[]])

                if not segments: continue

                merged_dialogue = []
                current_speaker = None
                current_text = ""

                for seg in segments:
                    speaker = "assistant" if seg.get('speaker_label') ==
"spk_0" else "user"
                    transcript = seg.get('transcript', "").strip()

                    if speaker == current_speaker:
                        current_text += " " + transcript
                    else:
                        if current_speaker is not None and
len(current_text) > 2:
                            merged_dialogue.append({"role":
current_speaker, "content": current_text.strip()})
                            current_speaker = speaker
                            current_text = transcript

                        if current_speaker is not None:
                            merged_dialogue.append({"role": current_speaker,
"content": current_text.strip()})

                all_calls.append({
                    "file": f_path.name,
                    "dialogue": merged_dialogue,
                    "turns": len(merged_dialogue)
                })
        except: continue

```

```

    return all_calls

all_calls = extract_all_data_merged(json_files)
df = pd.DataFrame(all_calls)

Fichiers trouvés dans tous les dossiers : 739

#On garde seulement les dialogues de bonne qualité
def is_valid_dialogue(dialogue):
    roles = set([t['role'] for t in dialogue])
    return len(dialogue) >= 3 and "assistant" in roles and "user" in roles

#application du filtre:
df['valid'] = df['dialogue'].apply(is_valid_dialogue)

df_clean = df[df['valid'] == True].copy()

# Supprimer l'intro automatique IVR (Interactive Voice Response)
def trim_ivr(dialogue):
    start_idx = 0
    for i, turn in enumerate(dialogue):
        if turn['role'] == "user" or "how may i help" in turn['content'].lower():
            start_idx = i
            break
    return dialogue[start_idx:]

df_clean['dialogue'] = df_clean['dialogue'].apply(trim_ivr)

print(f"Conversations après filtrage qualité : {len(df_clean)}")

Conversations après filtrage qualité : 538

def detect_intent_improved(dialogue):
    text = " ".join([t['content'].lower() for t in dialogue])

    # Cancellation (Annulation)
    if any(word in text for word in ["cancel", "unable to make it", "not coming"]):
        return "cancellation"

    # Rescheduling (Reporter)
    if any(word in text for word in ["reschedule", "change", "postpone", "another day"]):
        return "rescheduling"

    # Booking (Prise de RDV)
    booking_terms = ["book", "appointment", "schedule", "reserve", "set up"]

```

```

    if any(word in text for word in booking_terms):
        return "booking"

    return "other"

df_clean['intent'] =
df_clean['dialogue'].apply(detect_intent_improved)
print("\n Nouvel équilibre des intentions")
print(df_clean['intent'].value_counts())

Nouvel équilibre des intentions
intent
booking      340
other         82
cancellation  79
rescheduling  37
Name: count, dtype: int64

def preprocess_final(text):
    text = text.lower()
    # Anonymisation des ids de santé
    text = re.sub(r'\d{3}-\d{2}-\d{4}', '[HEALTH_ID]', text)
    # Anonymisation des num tel
    text = re.sub(r'\d{10}', '[PHONE]', text)
#Pour la Protection de la Vie Privée (RGPD).
# En remplaçant les données sensibles par des tokens génériques,
# on protège la confidentialité des individus tout en conservant la
structure du dialogue pour l'entraînement du modèle.

    text = re.sub(r'\b(uh|um|ah|euh|oh)\b', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

for dialogue in df_clean['dialogue']:
    for turn in dialogue:
        turn['content'] = preprocess_final(turn['content'])

print("Anonymisation et nettoyage terminés.")

Anonymisation et nettoyage terminés.

df_clean['intent'] =
df_clean['dialogue'].apply(detect_intent_improved)

import pandas as pd
import json

def balance_dataset(df):
    target_count = df[df['intent'] == 'booking'].shape[0]

```

```

    balanced_parts = []
    for intent in ['booking', 'other', 'cancellation',
'rescheduling']:
        intent_df = df[df['intent'] == intent]
        if not intent_df.empty:
            upsampled = intent_df.sample(target_count, replace=True,
random_state=42)
            balanced_parts.append(upsampled)

    df_result = pd.concat(balanced_parts).sample(frac=1,
random_state=42).reset_index(drop=True)
    return df_result

```

```
df_balanced = balance_dataset(df_clean)
```

```

print("--- Distribution après équilibrage ---")
print(df_balanced['intent'].value_counts())
print(f"Nombre total de lignes : {len(df_balanced)}")

```

```
--- Distribution après équilibrage ---
```

```

intent
booking      340
rescheduling  340
other         340
cancellation  340
Name: count, dtype: int64
Nombre total de lignes : 1360

```

```

import random
import re

```

```

DIVERSE_NAMES = ["James", "Mary", "Robert", "Patricia", "Jennifer",
"Linda", "Elizabeth", "Barbara", "Susan", "Jessica", "Sarah", "Karen",
"Nancy", "Lisa", "Betty", "Margaret", "Sandra", "Ashley", "Kimberly",
"Emily", "Donna", "Michelle", "Dorothy", "Carol", "Amanda", "Melissa",
"Deborah", "Stephanie", "Rebecca", "Sharon", "Laura", "Cynthia",
"Kathleen", "Amy", "Shirley", "Angela", "Helen", "Anna", "Brenda",
"Pamela", "Nicole", "Emma", "Samantha", "Katherine", "Christine",
"Debra", "Rachel", "Catherine", "Carolyn", "Janet", "Ruth", "Maria",
"Heather", "Diane", "Virginia", "Julie", "Joyce", "Victoria",
"Olivia", "Kelly", "Christina", "Lauren", "Joan", "Evelyn", "Judith",
"Megan", "Cheryl", "Andrea", "Hannah", "Martha", "Jacqueline",
"Frances", "Gloria", "Ann", "Teresa", "Kathryn", "Sara", "Janice",
"Jean", "Alice", "Madison", "Doris", "Abigail", "Julia", "Judy",
"Grace", "Denise", "Amber", "Marilyn", "Beverly", "Danielle",
"Theresa", "Sophia", "Marie", "Diana", "Brittany", "Natalie",
"Isabella", "Charlotte", "Rose", "Alexis", "Kayla"]
DIVERSE_CITIES = ["London", "Manchester", "Liverpool", "Bristol",
"Birmingham", "Leeds", "Sheffield", "Newcastle", "New York", "Los
Angeles", "Chicago", "Houston", "Phoenix", "Philadelphia", "San

```

```

Antonio", "San Diego", "Dallas", "San Jose", "Paris", "Marseille",
"Lyon", "Toulouse", "Nice", "Nantes", "Strasbourg", "Montpellier",
"Bordeaux", "Lille"]

def clean_and_replace_pii(text):
    """Remplace [pii] par un nom ou une ville aléatoire et nettoie le
    texte."""

    def random_replacement(match):
        return random.choice(DIVERSE_NAMES + DIVERSE_CITIES)

    text = re.sub(r'\[pii\]', random_replacement, text)

    # 3. Remplacer [PHONE] et [HEALTH_ID] par des nombres réalistes
    text = re.sub(r'\[PHONE\]', lambda x: f"{random.randint(100,999)}-
{random.randint(100,999)}-{random.randint(1000,9999)}", text)
    text = re.sub(r'\[HEALTH_ID\]', lambda x:
str(random.randint(100000, 999999)), text)

    text = text.replace(", ", ", ")
    text = re.sub(r'\s+', ' ', text)

    return text.strip()

import json

SHARED_SYSTEM_PROMPT = "You are a professional assistant. Your role is
to help clients book, cancel, or reschedule appointments with
precision and politeness."

def format_full_conversation(row):
    dialogue = row['dialogue']

    if not isinstance(dialogue, list) or len(dialogue) < 2:
        return None

    formatted_messages = [{"role": "system", "content":
SHARED_SYSTEM_PROMPT}]

    valid_turns = 0
    for turn in dialogue:
        content = clean_and_replace_pii(turn['content'])
        if len(content) > 3:
            formatted_messages.append({
                "role": turn['role'],
                "content": content
            })
            valid_turns += 1

    if valid_turns >= 2:

```

```

        return {"messages": formatted_messages}
    return None

formatted_series = df_balanced.apply(format_full_conversation,
axis=1).dropna()
formatted_data = formatted_series.tolist()

with open("cleaned_data.json", "w", encoding="utf-8") as f:
    json.dump(formatted_data, f, ensure_ascii=False, indent=4)

print("Succès ! Le fichier 'cleaned_data.json' est prêt.")
Succès ! Le fichier 'cleaned_data.json' est prêt.

with open("cleaned_data.jsonl", "w", encoding="utf-8") as f:
    for entry in formatted_data:
        f.write(json.dumps(entry, ensure_ascii=False) + "\n")
print(f"Terminé !")
print("cleaned_data.jsonl créé pour l'entraînement.")
Terminé !
cleaned_data.jsonl créé pour l'entraînement.

```